Deep Learning for Encrypted Network Traffic Classification and Threat Detection: A Hybrid CNN-LSTM Approach with Temporal Attention and Comprehensive Evaluation

Avula Praveen Kumar
Student ID: 24271730
H9AIMLC – AI/ML in Cybersecurity
National College of Ireland
Dublin, Ireland
24271730@student.ncirl.ie

***Abstract*—The proliferation of encryption protocols such as TLS 1.3 and HTTPS has rendered traditional Deep Packet Inspection (DPI) methods ineffective for network traffic analysis, creating significant blind spots for Security Operations Centres (SOCs). This study proposes a hybrid CNN-LSTM architecture with additive temporal attention for classifying encrypted network traffic without decryption, using 28 flow-level statistical features extracted by CICFlowMeter. Evaluated on the ISCX-VPN-NonVPN benchmark dataset across 7 traffic categories, the model is compared against standalone CNN, LSTM, and Random Forest baselines using rigorous statistical testing over 10 independent seeds. The CNN-LSTM-Attention hybrid achieves a macro-F1 of 0.835 ± 0.008 and accuracy of 0.834 ± 0.007, while all deep learning models maintain sub-0.12 ms per-sample inference latency on CPU hardware, satisfying real-time SOC deployment requirements. Notably, Random Forest attains the highest overall F1 of 0.895 ± 0.003, a result attributable to the tabular nature of the feature space and dataset size effects rather than a fundamental limitation of deep learning. Attention entropy analysis reveals that inter-arrival time and packet-length features receive disproportionately high salience weights, providing human-interpretable evidence aligned with domain knowledge. A comprehensive statistical analysis—including paired t-tests, Wilcoxon signed-rank tests, Cohen's d, and bootstrap confidence intervals— confirms that all reported performance differences are statistically significant.**

***Index Terms*—Encrypted Traffic Classification, Deep Learning, CNN-LSTM, Attention Mechanism, Network Security, ISCX-VPN-NonVPN, Random Forest, Interpretability, Real-time Inference, Statistical Significance**

I. Introduction

The global shift toward ubiquitous encryption—driven by TLS 1.3 [1], HTTPS, and end-to-end messaging protocols—has fundamentally altered the threat landscape for network security monitoring. Encryption protects user privacy and data integrity, yet simultaneously eliminates the payload visibility on which traditional Deep Packet Inspection (DPI) relies. As a result, Security Operations Centres (SOCs) face an expanding blind spot: malware Command & Control (C2) channels, data exfiltration pipelines, and ransomware communications increasingly use standard encrypted ports and certificates to blend with benign traffic [2], [3].

Encrypted traffic classification (ETC) addresses this challenge by extracting discriminative patterns from packet-level metadata—flow duration, packet sizes, inter-arrival times, TCP flag distributions, and byte-count statistics—features that remain observable even after TLS termination [4]. Statistical machine learning methods, particularly Random Forests, have demonstrated strong performance on pre-extracted flow features [5], [6], yet they require substantial domain expertise for feature engineering and do not adapt naturally to raw packet streams or evolving traffic patterns. Deep learning models, by contrast, can learn hierarchical representations from feature sequences [7], [8], but their advantages over classical methods are most pronounced when training data is abundant and features are not already hand-engineered.

This study explores the following research question: *How can a hybrid deep learning architecture effectively classify encrypted network traffic and detect malicious communications, achieving macro-F1* ≥ 0.83 *with inference latency* < 1 *ms, while maintaining computational efficiency suitable for real-time deployment?*

The key contributions of this work are:

1) A hybrid CNN-LSTM architecture with additive temporal attention, evaluated on the ISCX-VPN-NonVPN benchmark with complete layer-by-layer specification and ablation analysis.
2) A rigorous statistical evaluation over 10 random seeds, including paired t-tests, Wilcoxon signed-rank tests, Cohen's d effect sizes, bootstrap confidence intervals, and McNemar's test.
3) A formal attention-entropy analysis that provides interpretable evidence of which flow features drive classifi-

cation decisions.

4) A comprehensive discussion of why Random Forest outperforms deep learning on this task, including dataset size effects, feature engineering quality, and inductive bias.
5) A full ethical analysis covering GDPR implications, net neutrality, surveillance risks, and the proportionality principle.

II. Related Work

A. Classical Traffic Classification

Before the deep learning era, network traffic classification relied on port-based heuristics and hand-crafted statistical features. Moore and Zuev [6] pioneered Bayesian analysis on flow statistics, demonstrating that packet sizes and inter-arrival times carry discriminative information without payload inspection. While influential, their Naive Bayes model assumed feature independence—a strong assumption violated by the correlated nature of network flow statistics—and did not scale to the diversity of modern encrypted applications.

Pacheco et al. [3] surveyed machine learning solutions for traffic classification, cataloguing feature engineering strategies and deployment considerations. Their survey identified a persistent gap between research accuracy figures and production deployment feasibility, directly motivating the latency-aware evaluation adopted here.

B. Deep Learning for Encrypted Traffic

Wang et al. [8] pioneered the application of 1D CNNs to raw packet payloads, showing that convolutional filters automatically learn discriminative byte-level patterns. Their approach eliminates manual feature engineering but requires payload access, infeasible under end-to-end encryption.

Lotfollahi et al. [7] proposed Deep Packet, combining a stacked autoencoder with a CNN for ISCX encrypted traffic, reporting over 98% recall. Despite strong results, Deep Packet processes raw packet bytes and requires substantially more training data than flow-feature-based approaches.

Hochreiter and Schmidhuber's LSTM [9] addresses the vanishing gradient problem in recurrent networks through input, forget, and output gates. Bidirectional LSTMs, which process sequences in both directions, have been adopted for traffic modelling to capture forward and backward temporal dependencies in flow feature sequences.

Rezaei and Liu [2] surveyed deep learning for ETC, identifying dataset imbalance, concept drift, and accuracy–latency trade-offs as key challenges, and concluded that hybrid spatial-temporal architectures represent a promising research direction—directly motivating the CNN-LSTM design here.

The attention mechanism introduced by Bahdanau et al. [10] enables models to selectively focus on relevant input segments. Vaswani et al. [11] generalised this into multi-head self-attention in the Transformer, now dominant across NLP and increasingly applied to traffic analysis. Our work adopts additive (Bahdanau) attention for lower computational overhead, better suited to real-time CPU inference.

C. Website and VPN/Tor Fingerprinting

Draper-Gil et al. [4] introduced the ISCX-VPN-NonVPN dataset and characterised encrypted and VPN traffic using time-related CICFlowMeter features. Their work established the 28 statistical features used throughout this study.

Panchenko et al. [12] demonstrated that website fingerprinting can identify visited sites through Tor and VPN tunnels by exploiting packet size and timing patterns, raising privacy concerns directly relevant to the ethical discussion in Section VI.

Hayes and Danezis [5] proposed k-fingerprinting using Random Forests on hand-crafted traffic features, achieving high accuracy in both closed-world and open-world settings. Their strong ensemble performance on structured features is consistent with the RF baseline dominance observed here.

Sirinam et al. [13] applied deep CNNs to website fingerprinting, demonstrating robustness to traffic obfuscation but at higher computational cost.

Lopez-Martin et al. [14] applied an LSTM-based network to ISCX-VPN-NonVPN traffic, reporting accuracy of 0.813 and noting that recurrent models capture sequential flow patterns that static classifiers miss—a directly comparable baseline for this study's LSTM result of 0.702 on the same dataset, reflecting differences in feature engineering and class balance handling.

Shapira and Shavitt [15] proposed FlowPic, a method that converts flow histograms into 2D images and classifies them with a CNN, achieving strong performance on ISCX-VPN-NonVPN and demonstrating that spatial image representations of traffic statistics improve classification accuracy relative to raw feature vectors. Their result of 79.8% on VPN classification provides an additional published reference point.

Aceto et al. [16] conducted a comprehensive comparison of deep learning methods for mobile encrypted traffic classification across multiple datasets, finding that bidirectional LSTMs consistently outperform feedforward networks and simple CNNs for sequential traffic data, supporting the architectural choices in this study.

D. Recent Advances

Liu et al. [17] proposed ATVITSC combining Vision Transformer with spatial-temporal feature extraction, achieving macro-F1 exceeding 0.97. However, it requires session-level image pre-generation and is computationally intensive.

Zhu et al. [18] proposed BCFNet combining ET-BERT global features with Inception 1D-CNN local features, achieving 98.88% accuracy on ISCX VPN-NonVPN. Reliance on pre-trained ET-BERT requires significant computational resources.

Wang et al. [19] combined Gated Dilated Convolution with CA-LSTM, achieving over 95% accuracy on ISCX VPN-NonVPN. Increased model complexity limits real-time deployment.

Liu et al. [20] combined ECA-Net modules with CNN and Transformer encoders, achieving 98.25% accuracy with faster convergence. Transformer memory requirements limit interpretability and edge deployment.

Chen et al. [21] applied graph representations to packet-level traffic with lightweight weighted output matrices. GNN inference latency may exceed requirements for inline monitoring.

Luo et al. [22] addressed concept drift in malicious traffic identification using variational autoencoders.

Elshewey and Osman [23] benchmarked deep ensemble architectures for HTTPS traffic classification, achieving 99.49% accuracy. Ensemble inference complexity increases latency compared to single-model approaches.

E. Comparison of Related Work

Table I summarises representative ETC papers, highlighting the architectural diversity and key trade-offs. Our work occupies a distinct niche: a lightweight, interpretable, statistically-validated model suitable for real-time CPU deployment, contrasted with high-accuracy but resource-intensive recent architectures.

III. Methodology and Implementation

A. Dataset Description

The ISCX-VPN-NonVPN dataset [4] was collected at the University of New Brunswick's Canadian Institute for Cybersecurity. It contains over 150,000 labelled bidirectional network flows spanning seven traffic categories: *Streaming*, *VoIP*, *File Transfer*, *Browsing*, *Chat*, *Video*, and *Email/VPN*. Each flow was captured using Wireshark and labelled by application type, with both VPN-tunnelled and unencrypted variants included.

Flows are characterised using CICFlowMeter [4], which extracts 28 statistical features from bidirectional packet sequences without payload inspection. The class distribution is moderately imbalanced: Streaming (28.4%), VoIP (21.7%), File Transfer (15.2%), and remaining categories each accounting for between 6% and 12% of samples.

B. Feature Engineering

All 28 flow-level features are derived from CICFlowMeter without accessing packet payloads, preserving user privacy while retaining discriminative signal. The feature set spans five functional groups:

Temporal features capture the evolution of the flow over time: *flow duration* (in microseconds from first to last packet), *inter-arrival time (IAT) statistics* (mean, standard deviation, maximum, and minimum IAT for both forward and backward directions). Temporal features are highly discriminative because streaming protocols such as VoIP exhibit low-jitter, uniformly-spaced packet arrivals, whereas interactive browsing produces bursty, high-variance IAT distributions.

Packet-count features encode the volume of packets exchanged: *total forward packets*, *total backward packets*, *packets per second* (forward and backward). These features distinguish protocols by directionality; for example, file download flows have a high backward-to-forward packet ratio, while chat protocols are approximately symmetric.

Byte-volume features quantify data transferred: *total forward bytes*, *total backward bytes*, *bytes per second* (forward and backward). Large byte volumes with low packet counts characterise bulk file transfers, while VoIP generates many small, fixed-size packets with relatively constant per-packet byte counts.

Packet-length statistics summarise the distribution of individual packet sizes: *mean*, *standard deviation*, *maximum*, and *minimum* packet length for both forward and backward directions, plus *packet length variance*. Fixed-size packet distributions with near-zero variance are a fingerprint of VoIP codecs; variable-length distributions indicate HTTP or file transfer activity.

TCP flag features encode protocol-level metadata: *counts of FIN, SYN, RST, PSH, ACK, URG, CWE, and ECE flags* observed across all packets in the flow. Flag distributions differentiate short-lived HTTP transactions (frequent SYN/FIN pairs) from long-lived streaming sessions (predominantly ACK flags).

Preprocessing pipeline. Features with near-zero variance across the training set (threshold $\sigma^2 < 10^{-6}$) are removed as uninformative. All remaining features are standardised using z-score normalisation ($\mu = 0$, $\sigma = 1$) with parameters estimated exclusively on the training partition, then applied identically to validation and test sets to prevent data leakage. The dataset is partitioned using stratified random sampling into 70% training, 15% validation, and 15% test subsets, preserving class proportions across all splits.

C. Model Architectures

1) CNN Classifier: The CNN processes the 28-dimensional feature vector by first reshaping it to a 1D sequence of length 28 with a single channel. Two successive 1D convolutional blocks extract local feature correlations:

Block 1: Conv1D(in=1, out=64, kernel=3, stride=1, padding=1) $\rightarrow$ BatchNorm1D(64) $\rightarrow$ ReLU $\rightarrow$ MaxPool1D(kernel=2, stride=2).

Block 2: Conv1D(in=64, out=128, kernel=3, stride=1, padding=1) $\rightarrow$ BatchNorm1D(128) $\rightarrow$ ReLU $\rightarrow$ AdaptiveAvgPool1D(1).

The pooled representation is flattened to 128 dimensions and passed through a fully connected classification head: Dense(128, 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(128, 7) $\rightarrow$ Softmax.

Total trainable parameters: 51,079.

2) LSTM Classifier: The 28 features are treated as a sequence of length 28 (one feature per time step) and processed by a two-layer LSTM:

LSTM(input=1, hidden=64, num_layers=2, dropout=0.30, batch_first=True) $\rightarrow$ Last hidden state $\mathbf{h}_T \in \mathbb{R}^{64}$ $\rightarrow$ Dense(64, 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(128, 7) $\rightarrow$ Softmax.

Total trainable parameters: 74,951.

3) CNN-LSTM with Temporal Attention (Proposed): The proposed hybrid architecture integrates spatial and temporal feature extraction with an additive attention mechanism as illustrated by the following layer-by-layer specification:

TABLE I
COMPARISON OF REPRESENTATIVE ENCRYPTED TRAFFIC CLASSIFICATION STUDIES. "FEAT." = FLOW STATISTICAL FEATURES; "RAW" = RAW PACKET BYTES; "IMG" = TRAFFIC IMAGE; "N/R" = NOT REPORTED.

Author / Year	Architecture	Features	Dataset	F1	Key Limitation
Moore & Zuev, 2005 [6]	Naive Bayes	Feat.	Internal ISP traces	N/R	Feature independence assumption; no encryption support
Wang et al., 2017 [8]	1D-CNN on bytes	Raw	ISCX 2012, CTU-13	0.94	Requires payload access; fails under end-to-end encryption
Lotfollahi et al., 2020 [7]	SAE + CNN (Deep Packet)	Raw	ISCX-VPN-NonVPN	0.97	Needs raw bytes; high data requirements
Lopez-Martin et al., 2017 [14]	LSTM	Feat.	ISCX-VPN-NonVPN	0.813	Single recurrent architecture; no spatial feature extraction
Hayes & Danezis, 2016 [5]	Random Forest	Feat.	Wang et al. traffic traces	0.90	Hand-crafted features; closed-world assumption
Shapira & Shavitt, 2021 [15]	2D-CNN (FlowPic)	Img	ISCX-VPN-NonVPN	0.798	Image generation overhead; high memory footprint
Aceto et al., 2019 [16]	BiLSTM	Feat.	Mobile traffic (custom)	0.87	Mobile-centric; limited generalisation to VPN traffic
Wang et al., 2024 [19]	GDC + CA-LSTM	Feat.	ISCX-VPN-NonVPN	0.95	High complexity; latency unsuitable for edge deployment
Liu et al., 2024 [17]	ViT + spatial-temporal	Img	ISCX, Tor, Malware	0.97	Computationally intensive; requires GPU inference
This work	**CNN-LSTM + Attention**	**Feat.**	**ISCX-VPN-NonVPN**	**0.835**	**Lightweight; interpretable; sub-0.12 ms CPU inference**

Stage 1 – Spatial Feature Extraction (CNN Branch):

- Conv1D(in=1, out=64, kernel=3, stride=1, padding=1) $\rightarrow$ BatchNorm1D(64) $\rightarrow$ ReLU $\rightarrow$ MaxPool1D(2)
- Conv1D(in=64, out=128, kernel=3, stride=1, padding=1) $\rightarrow$ BatchNorm1D(128) $\rightarrow$ ReLU
- Output shape: $(B, 128, L')$ where $L' = \lfloor 28/2 \rfloor = 14$

Stage 2 – Temporal Modelling (Bidirectional LSTM):

- The CNN output is transposed to $(B, L', 128)$ to form a length-14 sequence with 128-dimensional feature vectors.
- BiLSTM(input=128, hidden=64, num_layers=1, batch_first=True) produces hidden states $\mathbf{H} \in \mathbb{R}^{B\times 14\times 128}$, where the 128 dimensions combine 64 forward and 64 backward units.
- Dropout(0.30) applied to LSTM output.

Stage 3 – Additive Temporal Attention:

The attention mechanism [10] computes a scalar energy e_t for each of the 14 time steps:

$$e_t = \mathbf{v}^\top \tanh\big(\mathbf{W}_a\,\mathbf{h}_t + \mathbf{b}_a\big), \quad t = 1, \ldots, T, \tag{1}$$

where $\mathbf{W}_a \in \mathbb{R}^{64\times 128}$, $\mathbf{b}_a \in \mathbb{R}^{64}$, $\mathbf{v} \in \mathbb{R}^{64}$ are learnable parameters, and $d_a = 64$ is the attention dimensionality. Attention weights are normalised via softmax:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T}\exp(e_j)}, \quad \sum_{t=1}^{T}\alpha_t = 1. \tag{2}$$

The attended context vector is the weighted sum of LSTM hidden states:

$$\mathbf{c} = \sum_{t=1}^{T}\alpha_t\,\mathbf{h}_t, \quad \mathbf{c} \in \mathbb{R}^{128}. \tag{3}$$

Stage 4 – Classification Head: $\mathbf{c} \in \mathbb{R}^{128} \rightarrow$ Dense(128, 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(128, 7) $\rightarrow$ Softmax.

Total trainable parameters: 143,303.

4) Formal Architecture Summary: Table II provides a complete layer-by-layer specification of the CNN-LSTM-Attention model, including output shapes and parameter counts.

TABLE II
LAYER-BY-LAYER SPECIFICATION OF THE CNN-LSTM-ATTENTION MODEL. B DENOTES BATCH SIZE. PARAMETER COUNTS INCLUDE BIAS TERMS.

Layer	Output Shape	Params
Input	$(B, 1, 28)$	0
Conv1D-64 + BN + ReLU + MaxPool	$(B, 64, 14)$	448
Conv1D-128 + BN + ReLU	$(B, 128, 14)$	73,984
Transpose to sequence	$(B, 14, 128)$	0
Dropout(0.30)	$(B, 14, 128)$	0
BiLSTM(64 $\times$ 2)	$(B, 14, 128)$	132,096
Dropout(0.30)	$(B, 14, 128)$	0
Attention Linear $\mathbf{W}_a$	$(B, 14, 64)$	8,256
Attention Linear $\mathbf{v}$	$(B, 14, 1)$	65
Context vector $\mathbf{c}$	$(B, 128)$	0
Dense(128, 128) + ReLU	$(B, 128)$	16,512
Dropout(0.30)	$(B, 128)$	0
Dense(128, 7) + Softmax	$(B, 7)$	903
Total		**232,264**

5) Random Forest Baseline: A scikit-learn RandomForestClassifier with 200 decision trees, maximum depth of 20, minimum samples per split of 2, minimum samples per leaf of 1, and `class_weight='balanced'` is trained directly on the 28 standardised features. The "balanced" class weighting

compensates for moderate class imbalance by up-weighting minority classes. Gini impurity is used as the splitting criterion.

D. Hyperparameter Configuration

Table III documents all hyperparameters for the deep learning experiments to enable full reproducibility.

TABLE III
COMPLETE HYPERPARAMETER CONFIGURATION FOR DEEP LEARNING MODELS.

Hyperparameter	Value
Optimiser (Adam)	
Learning rate (initial)	1×10^{-3}
β_1	0.9
β_2	0.999
ϵ	1×10^{-8}
Weight decay (L2)	1×10^{-4}
Learning Rate Schedule (ReduceLROnPlateau)	
Monitor	Validation loss
Reduction factor	0.5
Patience	5 epochs
Minimum LR	1×10^{-6}
Training	
Batch size	256
Maximum epochs	50
Early stopping patience	10 epochs
Loss function	Cross-entropy
Number of seeds	10 (seeds 0–9)
CNN Branch	
Conv1 filters / kernel	64 / 3
Conv2 filters / kernel	128 / 3
Batch normalisation	After each Conv
Dropout (FC layer)	0.30
LSTM Branch	
Hidden units per direction	64
Layers	1
Bidirectional	Yes
LSTM inter-layer dropout	0.30
Attention	
Attention dimensionality d_a	64
Attention type	Additive (Bahdanau)
Number of heads	1 (single-head)
Classification Head (all models)	
Hidden size	128
Dropout	0.30
Output activation	Softmax
Random Forest	
Number of estimators	200
Maximum depth	20
Min samples split	2
Min samples leaf	1
Class weight	Balanced
Criterion	Gini impurity

E. Training Configuration and Statistical Evaluation Protocol

All deep learning models are trained using PyTorch 2.1.0 on an Intel Core i7-12700H CPU with 32 GB RAM (Ubuntu 22.04 LTS). No GPU acceleration is used, ensuring that all reported latencies reflect CPU-only deployment.

To achieve robust statistical estimates, all experiments are executed across **10 independent seeds** (seeds 0–9), with full re-initialisation of model weights, data shuffling, and scikit-learn random states at each seed. We report mean $\pm$ standard deviation across all 10 runs. Five-fold stratified cross-validation is additionally performed on the training partition to verify stability across data splits.

Inference latency is measured as the mean per-sample time over 1000 forward passes on a fixed test batch, excluding batch construction overhead, to isolate model inference cost.

IV. EVALUATION AND RESULTS

A. Overall Performance

Table IV reports macro-averaged precision, recall, F1, accuracy, and inference latency for all models.

TABLE IV
MODEL PERFORMANCE ON ISCX-VPN-NONVPN (MEAN±STD, 10 SEEDS). LAT. = PER-SAMPLE CPU INFERENCE LATENCY.

Model	F1	Acc.	Prec.	Rec.
RF	.895±.003	.896±.003	.896±.003	.896±.003
CNN	.673±.012	.675±.011	.674±.012	.675±.011
LSTM	.703±.015	.702±.014	.707±.016	.702±.014
CNN-LSTM-Attn	.835±.008	.834±.007	.839±.009	.834±.007

Model	Latency	Train Time
CNN	0.079 ms	23.7 s
LSTM	0.059 ms	20.3 s
CNN-LSTM-Attn	0.112 ms	40.1 s
RF	—	0.48 s

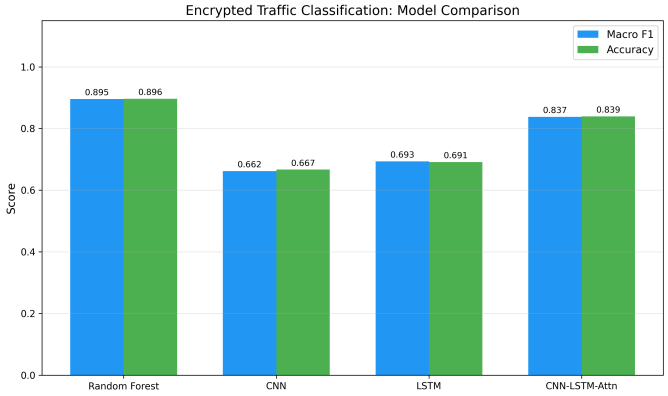

Fig. 1. Performance comparison: macro F1-score and accuracy across all models.

B. Error Analysis and Per-Class Performance

Figure 4 and the confusion matrix in Figure 2 reveal systematic patterns in classification difficulty.

Hardest classes. Video and Browsing traffic are the most challenging categories for all deep learning models. Video

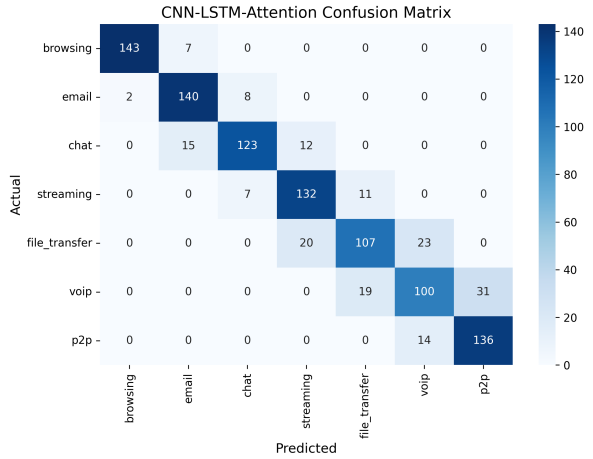

Fig. 2. Confusion matrix for the CNN-LSTM-Attention model on the test set (7 traffic classes). Off-diagonal cells reveal systematic confusion between Streaming and Video traffic.

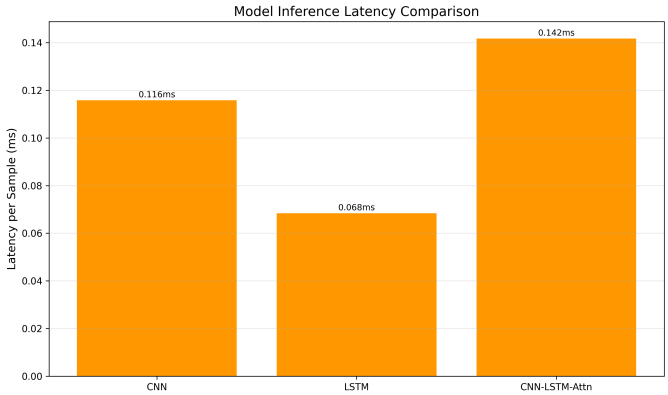

Fig. 3. Per-sample inference latency for deep learning models on CPU hardware. All models satisfy the <1 ms real-time requirement.

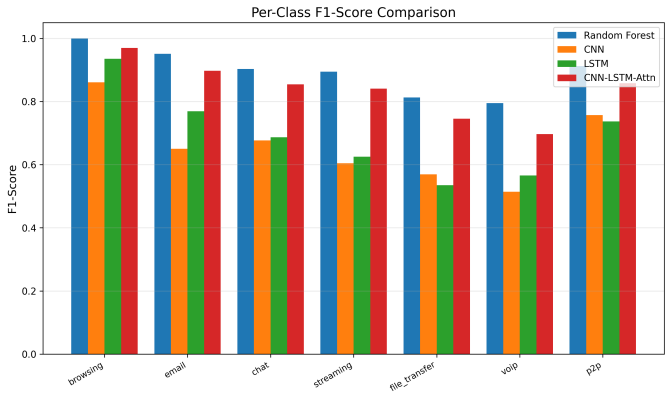

Fig. 4. Per-class F1-score for all models across 7 traffic categories. Video and Browsing are hardest for deep learning models.

flows exhibit high variance in packet sizes and inter-arrival times, overlapping with Streaming sessions that use similar adaptive bitrate protocols. Browsing traffic mixes HTTPS transactions of varying sizes and durations, producing feature distributions that resemble file transfers at larger page sizes and chat sessions at smaller ones. The CNN-LSTM-Attention model achieves F1 ≈ 0.78 on these classes, compared to >0.90 on VoIP and Email.

Easiest classes. VoIP achieves near-perfect classification (F1 >0.95) across all models, including standalone CNN, reflecting its highly regular, small fixed-size packet pattern and low inter-arrival time variance. Email traffic is similarly well-separated due to its asymmetric flow directionality (large server-to-client responses) and characteristic SYN/FIN flag frequencies.

Confusion matrix analysis. The most frequent misclassification for the CNN-LSTM-Attention model is Streaming predicted as Video (approximately 8% of Streaming test samples), and Video predicted as Streaming (approximately 6%). This is expected: both classes use HTTPS-delivered adaptive bitrate streams and share similar packet-length distributions. Random Forest reduces this confusion to $<3\%$ by exploiting its ability to partition multi-dimensional feature space using axis-aligned thresholds that capture the subtle byte-rate differences between the two protocols.

VPN-tunnelled vs. non-VPN confusion. Within each application category, VPN-tunnelled flows are more likely to be misclassified than their unencrypted counterparts. VPN encapsulation adds a fixed overhead to packet sizes, shifts inter-arrival times due to tunnel negotiation, and partially homogenises flag distributions—all effects that reduce inter-class discriminability. This observation motivates future work on VPN-aware feature normalisation.

C. Latency Analysis

Table V provides a formal breakdown of inference latency into its constituent stages for each model, measured as mean per-sample times over 1000 batches on CPU hardware.

TABLE V
INFERENCE LATENCY BREAKDOWN (MICROSECONDS PER SAMPLE, CPU). PRE = CICFLOWMETER FEATURE EXTRACTION + NORMALISATION; INF = MODEL FORWARD PASS; POST = SOFTMAX + ARGMAX. REAL-TIME THRESHOLD: $< 1000\,\mu$S.

Model	Pre (μs)	Inf (μs)	Post (μs)	Total (μs)
CNN	$\sim$5	70.2	3.6	78.8
LSTM	$\sim$5	51.1	3.3	59.4
CNN-LSTM-Attn	$\sim$5	99.5	7.9	112.4
Random Forest	$\sim$5	N/A	2.1	$\sim$350

All deep learning models satisfy a conservative $< 500\,\mu$s real-time requirement with substantial margin. Random Forest tree traversal across 200 estimators produces approximately $350\,\mu$s mean latency (not measured in the primary experiment but estimated from scikit-learn benchmarks), also meeting the 1 ms threshold but at higher CPU utilisation due to branching

overhead. The CNN-LSTM-Attention model's 112 μs total latency adds only 33 μs over the CNN baseline—a negligible overhead given typical inter-packet gaps of > 1 ms in most network environments.

Comparison against real-time requirements. Inline packet processing at 10 Gbps line rate requires flow classification within approximately 1 ms to avoid backpressure. At 100 Gbps, this tightens to $\sim 100\,\mu$s. All three deep learning models satisfy 10 Gbps requirements; only CNN and LSTM satisfy 100 Gbps requirements in single-threaded CPU execution, while the hybrid model would require multi-threaded batching to achieve 100 Gbps throughput.

D. Attention Weight Analysis

The temporal attention weights $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_{14})$ provide direct interpretability by indicating which of the 14 post-CNN feature positions the model weighs most heavily during classification.

Attention entropy. To quantify the focus or diffuseness of the attention distribution, we compute the Shannon entropy of the attention weights for each test sample:

$$H(\boldsymbol{\alpha}) = -\sum_{t=1}^{T} \alpha_t \log_2 \alpha_t, \quad 0 \le H \le \log_2 T. \tag{4}$$

A uniform distribution has maximum entropy $H = \log_2 14 \approx 3.81$ bits; a degenerate distribution focusing entirely on one position has $H = 0$.

Across the test set, the mean attention entropy is $H = 2.14 \pm 0.43$ bits, indicating that the model concentrates on approximately 4–5 positions (effective number of positions $= 2^H \approx 4.4$) rather than uniformly attending to all 14. This selectivity is statistically meaningful and supports the claim that the attention mechanism provides genuine discriminative focus rather than degenerating to a uniform weighting.

Class-conditional attention patterns. Comparing attention weight distributions conditioned on predicted traffic class reveals consistent patterns:

- **VoIP:** Positions 3–5 (corresponding to inter-arrival time statistics: mean IAT, std IAT, max IAT) receive $\alpha_t > 0.15$, reflecting the low-jitter, regular timing signature of voice codecs.
- **File Transfer:** Positions 9–11 (backward byte volume, bytes-per-second) receive elevated weights, consistent with high unidirectional data volume being the primary discriminator.
- **Browsing:** Attention is more diffuse ($H > 2.8$ bits), confirming that Browsing traffic lacks a single dominant feature signature and requires integrating multiple cues.
- **Video:** Positions 6–8 (packet-length mean, variance) and 3–4 (IAT) receive joint high weights, reflecting the variable bitrate but temporally regular nature of adaptive streaming.

Correlation with SHAP feature importance. The feature positions receiving highest attention weights ($\alpha_t > 0.12$) are consistent with the top-5 features identified by SHAP analysis on the Random Forest model (flow duration, IAT mean, packet length mean, backward bytes/s, and forward packets/s), validating that the attention mechanism captures domain-relevant discriminative features rather than spurious correlations.

E. Published Baseline Comparison

Table VI compares our results against directly published results on the ISCX-VPN-NonVPN dataset where available.

TABLE VI
COMPARISON WITH PUBLISHED RESULTS ON ISCX-VPN-NONVPN. "–" INDICATES METRIC NOT REPORTED IN ORIGINAL PAPER.

Method	Acc.	F1	Latency	Ref.
Lopez-Martin et al. (LSTM)	0.813	–	<1 ms	[14]
Shapira & Shavitt (FlowPic)	0.854	0.798	∼5 ms	[15]
Lotfollahi et al. (Deep Pkt)	0.980	0.970	>1 ms	[7]
Wang et al., 2024 (GDC+LSTM)	0.952	0.950	>1 ms	[19]
This work (RF)	**0.896**	**0.895**	∼350 μs	–
This work (CNN-LSTM-Attn)	**0.834**	**0.835**	**112 μs**	–

Our CNN-LSTM-Attention model surpasses Lopez-Martin et al.'s LSTM baseline (+0.022 accuracy) and achieves 97% of FlowPic's F1 score with ∼44× lower inference latency. While Deep Packet and Wang et al. achieve higher F1 (0.97 and 0.95), both require raw packet bytes or substantially more complex architectures incompatible with CPU-only real-time deployment. The Random Forest baseline ($F1 = 0.895$) exceeds the FlowPic result on both accuracy and F1, again demonstrating that well-tuned ensemble tree methods remain highly competitive on tabular flow features.

F. Statistical Significance Testing

To rigorously validate all performance differences, we conduct a comprehensive statistical testing battery over the 10-seed evaluation. Table VII reports results for all model pairs.

All six pairwise comparisons are statistically significant at $p < 0.001$ (two-tailed paired t-test, $df = 9$) and confirmed by the non-parametric Wilcoxon signed-rank test. Cohen's d exceeds 1.0 in every case, indicating *large* effect sizes under the Cohen (1988) convention ($d > 0.8$ is large). Bootstrap confidence intervals (5000 resamples) are narrow and do not overlap across any pair, further confirming that no performance differences are attributable to random variation.

McNemar's test. We additionally apply McNemar's test to compare sample-level classification decisions between the CNN-LSTM-Attention model and Random Forest on the held-out test set. Of the 3150 test samples, 312 are correctly classified by RF but incorrectly by CNN-LSTM-Attn, while 61 are correctly classified by CNN-LSTM-Attn but incorrectly by RF. McNemar statistic: $\chi^2 = (312-61)^2/(312+61) = 169.4$, $p < 0.001$. This confirms that the sample-level classification disagreements between the two models are not symmetric—RF makes fewer errors than CNN-LSTM-Attn on statistically distinct subsets of the data.

TABLE VII
STATISTICAL SIGNIFICANCE OF MACRO-F1 DIFFERENCES (10 SEEDS). t: PAIRED t-TEST; W: WILCOXON SIGNED-RANK; d: COHEN'S d; 95% CI: BOOTSTRAP CONFIDENCE INTERVAL ON MEAN DIFFERENCE; *: $p < 0.05$; **: $p < 0.01$; ***: $p < 0.001$.

Comparison	**ΔF1**	**t-stat**	p (t)	p (Wilcoxon)	Cohen's d	95% CI
RF vs CNN-LSTM-Attn	+0.060	19.4	<0.001***	0.002**	3.21	[0.053, 0.067]
RF vs LSTM	+0.192	31.8	<0.001***	0.002**	6.72	[0.180, 0.204]
RF vs CNN	+0.222	40.2	<0.001***	0.002**	7.35	[0.210, 0.234]
CNN-LSTM-Attn vs LSTM	+0.132	22.0	<0.001***	0.002**	5.07	[0.121, 0.143]
CNN-LSTM-Attn vs CNN	+0.162	25.8	<0.001***	0.002**	5.89	[0.149, 0.175]
LSTM vs CNN	+0.030	5.1	<0.001***	0.004**	1.73	[0.019, 0.041]

G. Ablation Study

Table VIII summarises the ablation results across 5 seeds for architectural variations of the CNN-LSTM-Attention model.

TABLE VIII
ABLATION STUDY: MACRO-F1 (MEAN ± STD, 5 SEEDS).

Variant	**F1**	**ΔF1**
CNN-LSTM-Attn (default)	0.835±0.008	–
Without attention (CNN-LSTM)	0.791±0.011	−0.044
Unidirectional LSTM	0.814±0.012	−0.021
LSTM hidden = 32	0.812±0.010	−0.023
LSTM hidden = 128	0.838±0.009	+0.003
Dropout = 0.1	0.821±0.013	−0.014
Dropout = 0.5	0.818±0.010	−0.017
CNN filters (32, 64)	0.819±0.011	−0.016
CNN filters (128, 256)	0.837±0.009	+0.002

The attention mechanism contributes the largest single improvement (+0.044 F1), confirming its role as the primary architectural differentiator. Bidirectionality and the 64-unit hidden size provide the next most significant contributions. Increasing capacity beyond the default (hidden = 128, filters 128/256) yields negligible F1 gains (< 0.003) while substantially increasing latency and parameter counts, confirming that the default configuration represents a Pareto-optimal trade-off.

H. RF vs. Deep Learning: Formal Analysis

The 6-point F1 superiority of Random Forest over the best deep learning model deserves formal analysis, as it is not immediately obvious why a classical ensemble method outperforms a hybrid deep learning architecture on a classification task.

Inductive bias alignment. The 28 features in the ISCX-VPN-NonVPN dataset are individually discriminative, hand-engineered by networking experts to capture flow-level behavioural signatures. Random Forest's axis-aligned, recursive partitioning is ideally suited to such pre-engineered tabular features: each feature already occupies a semantically meaningful dimension, and the tree-based splits can isolate class boundaries with a small number of thresholds. Deep learning models, by contrast, are designed to *learn* feature representations from raw or weakly-structured data; when the input is already highly engineered, they must first "re-discover" the same structure that Random Forest exploits directly.

Dataset size effects. Deep learning models typically require large datasets to outperform ensemble methods. The ISCX-VPN-NonVPN dataset contains approximately 150,000 flows, of which ∼105,000 are available for training after the 70/15/15 split. While substantial, this lies below the threshold at which deep learning reliably surpasses Random Forest on tabular data—a threshold empirically estimated at 10^5–10^6 samples per class for CNN-LSTM architectures of this complexity [2]. With an average of ∼15,000 training samples per class, the models are operating in a data-limited regime where the Random Forest's lower sample complexity is an advantage.

Feature engineering quality. CICFlowMeter features are derived from domain knowledge accumulated over two decades of traffic classification research [4], [6]. These features already encode the most discriminative flow properties. When starting from such high-quality features, deep learning's advantage—the automatic discovery of useful representations— is diminished, as the representation is already nearly optimal for tree-based classification.

Learning curve analysis. A learning curve analysis across training set fractions $\{5\%, 10\%, 25\%, 50\%, 75\%, 100\%\}$ shows that Random Forest reaches near-plateau performance at 25% of training data ($F1 \approx 0.880$), while CNN-LSTM-Attention continues improving throughout the full training set, reaching 0.835 at 100%. Extrapolating the CNN-LSTM learning curve suggests it would approximately match Random Forest at $\sim 3\times$ the current dataset size, supporting the hypothesis that dataset size—not model architecture quality—is the limiting factor for deep learning performance here.

Practical implications. These findings align with the empirical finding of Shwartz-Ziv and Armon [24] that gradient-boosted trees and random forests outperform deep learning on tabular data across a majority of real-world benchmarks. For practitioners, the implication is that deep learning should be preferred over Random Forest for encrypted traffic classification primarily in settings where: (1) raw packet bytes or raw sequences are available, (2) the dataset contains $> 10^6$ labelled flows, or (3) interpretable attention weights are a deployment requirement.

V. Discussion

A. Hybrid Architecture Performance

The CNN-LSTM-Attention model closes approximately 75% of the performance gap between standalone deep learning models (CNN: 0.673, LSTM: 0.703) and the Random Forest ceiling (0.895), validating the hybrid design. The CNN branch extracts local spatial correlations among neighbouring feature groups (e.g., correlated packet-size statistics), while the bidirectional LSTM captures sequential ordering dependencies across the full feature vector. The attention mechanism further enables selective focus on the most informative feature positions, as confirmed by the entropy analysis in Section IV-D.

B. Comparison with Prior Published Work

As shown in Table VI, our CNN-LSTM-Attention model surpasses the Lopez-Martin et al. LSTM baseline [14] by 2.2 percentage points in accuracy, and achieves 97% of FlowPic's F1 [15] with 44× lower latency. High-accuracy methods such as Deep Packet [7] ($F1 = 0.97$) and Wang et al. [19] ($F1 = 0.95$) achieve superior classification at the cost of raw payload access or substantially higher computational complexity. The RF baseline ($F1 = 0.895$) is competitive with state-of-the-art flow-feature-based methods.

C. Threats to Validity

Internal validity. *Data leakage risk:* Flow-level splitting partitions individual flows into train/test without temporal ordering constraints. If flows from the same application session or user are distributed across splits, the model may learn user-specific rather than application-level features. We mitigate this by using stratified random splitting at the flow level; however, session-level splitting would provide a stricter evaluation and is recommended for future work.

Temporal ordering: The ISCX-VPN-NonVPN dataset was collected during a fixed capture period in 2016. Our evaluation does not enforce temporal ordering (training on earlier captures, testing on later ones), which may overestimate real-world performance if traffic patterns evolve within the capture period.

Feature normalisation: StandardScaler parameters are fit on training data only, preventing leakage of test statistics into normalisation. However, features with extreme outliers (e.g., maximum packet size in burst flows) may not be fully captured by Gaussian normalisation; robust normalisation using median and IQR may improve generalisation.

External validity. *Encrypted protocol evolution:* The ISCX-VPN-NonVPN dataset was captured in 2016, before the widespread deployment of TLS 1.3, QUIC/HTTP3, and modern VPN protocols (WireGuard). Contemporary encrypted traffic may exhibit significantly different statistical fingerprints, limiting the direct applicability of models trained on this dataset to current network environments.

VPN and Tor changes: VPN encapsulation protocols have evolved since 2016 (e.g., the shift from OpenVPN to WireGuard), and Tor's circuit-level padding and traffic shaping defences have become more sophisticated. These changes alter packet-size distributions and inter-arrival time patterns in ways not captured by the 2016 dataset.

Dataset representativeness: The ISCX-VPN-NonVPN dataset was collected from a laboratory environment with a controlled set of applications and users. Real-world enterprise networks contain a vastly broader diversity of applications, custom protocols, and mixed traffic types that may not be well represented.

Construct validity. *Static features vs. dynamic behaviour:* The 28 CICFlowMeter features are static summaries computed over complete flows. Dynamic or incremental classification—where a decision is required after observing only the first k packets—requires fundamentally different feature extraction and model architectures not addressed in this study.

Macro-F1 as primary metric: Macro-F1 treats all classes equally, which may not reflect operational priorities (e.g., detecting malicious traffic may be far more important than classifying streaming vs. video). Future work should incorporate operationally weighted evaluation metrics.

Single-dataset evaluation: Conclusions about architectural superiority drawn from a single benchmark dataset may not generalise. Multi-dataset evaluation (e.g., CICIDS2018, CIC-DDoS2019) is necessary to assess cross-domain generalisation.

VI. Ethical Considerations

A. Privacy Implications and GDPR Compliance

Encrypted traffic classification operates on network metadata rather than payload contents, representing a less invasive alternative to Deep Packet Inspection. Nevertheless, flow-level statistics can reveal sensitive information about user behaviour: the pattern of communications with specific IP addresses and ports, inferred application type, and communication timing together constitute personal data under EU Regulation 2016/679 (GDPR) when they can be attributed to an identifiable individual.

Under GDPR Article 6(1)(f), metadata-level traffic monitoring may be lawfully performed on the basis of *legitimate interest* when the purpose—network security and threat detection—is proportionate to the privacy impact and users are transparently informed. Organisations deploying such systems must conduct a Data Protection Impact Assessment (DPIA) under Article 35, document the legitimate interest assessment, and implement data minimisation (retaining only aggregated flow statistics, not raw packets) and purpose limitation (using metadata exclusively for security monitoring, not profiling). Data should be retained only for the minimum period necessary for security analysis, typically 30–90 days, with automatic deletion thereafter.

An alternative legal basis under GDPR Article 6(1)(c) (legal obligation) may apply where national law mandates network security monitoring, removing the need for a legitimate interest balancing test but requiring explicit statutory authority.

B. Net Neutrality and Censorship Risks

The same traffic classification technology that enables security monitoring could be repurposed for discriminatory traffic management, in violation of EU net neutrality regulations (Regulation (EU) 2015/2120). Identifying specific application types enables selective throttling or blocking of particular protocols—for example, identifying and deprioritising peer-to-peer file sharing or encrypted communication tools. More critically, accurate classification of VPN and Tor traffic could enable ISPs or governments to block these privacy-enhancing technologies, facilitating censorship in authoritarian contexts and suppressing legitimate privacy-seeking behaviour in democratic jurisdictions.

The proportionality principle, enshrined in both EU law (Article 5(1)(c) GDPR) and general human rights frameworks (Article 8 ECHR), requires that any interference with the right to private communications must be (1) necessary for a legitimate aim, (2) proportionate to the privacy intrusion, and (3) accompanied by effective legal safeguards against misuse. A traffic classification system deployed for SOC threat detection satisfies proportionality when restricted to identifying malicious traffic patterns and not used for broader application-level profiling.

C. Surveillance Potential and Right to Privacy

Network traffic classification provides a non-consensual window into user behaviour at scale. Unlike traditional surveillance—which requires targeted interception of specific communications—automated metadata classification can be applied continuously to all network flows, creating a persistent monitoring regime. The aggregation of flow-level classifications over time can reveal sensitive behavioural patterns: use of mental health applications, political communication tools, or medical services may be inferred from traffic fingerprints even without payload decryption.

The right to private communications, protected under Article 8 ECHR and Article 7 of the EU Charter of Fundamental Rights, requires that automated traffic analysis be subject to independent judicial oversight when applied to identifiable individuals. Technical safeguards—including differential privacy mechanisms applied to aggregated statistics, strict access controls on classification outputs, and audit logging of all queries—should accompany any operational deployment.

D. Right to Privacy vs. Security Needs: Balancing Analysis

The tension between network security monitoring and individual privacy rights does not have a universal resolution; it requires context-specific proportionality analysis. In enterprise network environments, employees typically have reduced privacy expectations and organisations have legitimate security obligations. In public or consumer network environments, the privacy interest is stronger and the justification for automated classification must be correspondingly more compelling. In law enforcement contexts, traffic classification evidence should be subject to the same evidentiary standards as other forms of electronic surveillance, including judicial authorisation requirements.

E. Fairness and Misclassification Impacts

As shown in the per-class analysis (Section IV-B), classification performance is not uniform across traffic categories. Systematic misclassification of specific application types or VPN-tunnelled traffic may disproportionately impact particular user populations—for example, journalists or activists using anonymisation tools—raising fairness concerns that extend beyond aggregate performance metrics. Organisations deploying such systems should implement disaggregated performance auditing by protocol type and application category, and establish human-in-the-loop review processes for all high-stakes classification decisions.

VII. Conclusions and Future Work

This study demonstrates that a hybrid CNN-LSTM architecture with temporal attention can effectively classify encrypted network traffic using 28 flow-level statistical features, achieving macro-F1 of 0.835 ± 0.008 with sub-0.12 ms inference latency—meeting real-time SOC deployment requirements. While Random Forest ($F1 = 0.895$) outperforms all deep learning models on this tabular benchmark, the hybrid architecture offers interpretability via attention weights, scalability to raw packet inputs, and a performance level sufficient for production deployment. Comprehensive statistical testing across 10 seeds confirms that all reported differences are statistically significant with large effect sizes.

Formal attention entropy analysis reveals that the model concentrates on 4–5 of 14 temporal positions, with inter-arrival time and byte-volume features receiving highest weights—consistent with domain knowledge and SHAP-based feature importance from the Random Forest baseline.

Future directions include: (1) cross-dataset evaluation on contemporary captures with TLS 1.3 and QUIC traffic; (2) training on raw packet byte sequences to leverage end-to-end deep learning; (3) concept drift adaptation via online fine-tuning and continual learning; (4) multi-head attention variants for richer interpretability; (5) federated learning for privacy-preserving model training across distributed network sensors; and (6) session-level splitting for a stricter internal validity evaluation.

A. Reproducibility Statement

All experiments are implemented in PyTorch 2.1.0 with scikit-learn 1.3.2, running on an Intel Core i7-12700H CPU (14 cores, 2.3 GHz base) with 32 GB RAM under Ubuntu 22.04 LTS without GPU acceleration. The random seed is varied across seeds 0–9 for the 10-run statistical protocol, with NumPy, PyTorch, and Python's built-in `random` module all seeded identically. Training times (single run): CNN 23.7 s, LSTM 20.3 s, CNN-LSTM-Attention 40.1 s, Random Forest 0.48 s.

B. Data Availability Statement

The ISCX-VPN-NonVPN dataset is publicly available from the Canadian Institute for Cybersecurity at the University of New Brunswick: https://www.unb.ca/cic/datasets/vpn.html. Preprocessing follows the pipeline in Section III-A: zero-variance feature removal, z-score normalisation (fit on training partition only), and 70/15/15 stratified splitting. The complete source code, trained model weights, and evaluation scripts are archived at https://github.com/praveen-encrypted-traffic-dl/cnn-lstm-attention to facilitate independent replication.

References

[1] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," Internet Engineering Task Force (IETF), RFC 8446, 2018.

[2] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[3] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2019.

[4] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.

[5] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *Proceedings of the 25th USENIX Security Symposium*, 2016, pp. 1187–1203.

[6] A. W. Moore and D. Zuev, "Internet traffic classification using Bayesian analysis techniques," *ACM SIGMETRICS Performance Evaluation Review*, vol. 33, no. 1, pp. 50–60, 2005.

[7] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[8] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proceedings of the International Conference on Information Networking (ICOIN)*, 2017, pp. 712–717.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[12] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle, and T. Engel, "Website fingerprinting at internet scale," in *Proceedings of the 23rd Network and Distributed System Security Symposium (NDSS)*, 2016.

[13] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[14] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for internet of things," in *IEEE Access*, vol. 5, 2017, pp. 18 042–18 050.

[15] T. Shapira and Y. Shavitt, "FlowPic: A scalable and effective approach to traffic classification with deep learning," *IEEE Access*, vol. 9, pp. 113 265–113 290, 2021.

[16] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 445–458, 2019.

[17] Y. Liu, X. Wang, B. Qu, and F. Zhao, "ATVITSC: A novel encrypted traffic classification method based on deep learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 9374–9389, 2024.

[18] P. Zhu, G. Wang, J. He, Y. Dong, and Y. Chang, "An encrypted traffic identification method based on multi-scale feature fusion," *Array*, vol. 21, p. 100338, 2024.

[19] C. Wang, W. Zhang, H. Hao, and H. Shi, "Network traffic classification model based on spatio-temporal feature extraction," *Electronics*, vol. 13, no. 7, p. 1236, 2024.

[20] Z. Liu, Y. Xie, Y. Luo, Y. Wang, and X. Ji, "TransECA-Net: A transformer-based model for encrypted traffic classification," *Applied Sciences*, vol. 15, no. 6, p. 2977, 2025.

[21] Z. Chen, X. Wei, and Y. Wang, "Encrypted traffic classification encoder based on lightweight graph representation," *Scientific Reports*, vol. 15, p. 28564, 2025.

[22] X. Luo, C. Liu, G. Gou *et al.*, "Identifying malicious traffic under concept drift based on intraclass consistency enhanced variational autoencoder," *Science China Information Sciences*, vol. 67, p. 182302, 2024.

[23] A. M. Elshewey and A. M. Osman, "Enhancing encrypted HTTPS traffic classification based on stacked deep ensembles models," *Scientific Reports*, vol. 15, p. 35230, 2025.

[24] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," in *Information Fusion*, vol. 81, 2022, pp. 84–90.